Midterm exam review

Wednesday, June 10, 2026

Today: Wednesday June 10

Plan for today

- Exam is **tomorrow, Thursday June 11, in class**
- Review
- Practice problems

Helpful links

- Syllabus · Schedule · *R4DS* Ch. 1–5 · *DC* Ch. 3–5, 11–12

The exam

Logistics

- **When:** Thursday June 11, start of class (9:35 a.m.)
- **How long:** 75 minutes
- **Points:** 100 total, 8 parts (A-H)
- **Closed book**

You may assume `tidyverse` and `palmerpenguins` are loaded. You'll see these datasets: `penguins`, `mpg`, `diamonds`, `msleep`.

Format breakdown:

Part	Type	Points
A	Multiple choice (12 items)	24
B	Predict the output	12
C	Code anatomy	10

Part	Type	Points
D	Code writing	20
E	Position adjustments	8
F	Facets and scales	6
G	Tidy data	8
H	Short answer & debugging	12

Study advice

The exam tests whether you can **read, write, and reason about** code. Some study suggestions:

- For multiple choice, know *why* each answer is correct or incorrect, not just the letter
- For predict-the-output, run the expressions yourself and understand the rule
- For code writing, practice writing from scratch instead of just reading
- For debugging think about "what rule does this code violate?"

R basics

Assignment and objects

The standard assignment operator is `<-`. `=` works too, but `<-` is the course convention and the idiomatic R style.

```
x <- 7.3 # preferred
x
```

```
[1] 7.3
```

```
x = 8.1 # works, not preferred
x
```

```
[1] 8.1
```

```
1.2 -> x # also works, really not preferred
x
```

```
[1] 1.2
```

`==` is a **comparison**, not assignment; it tests whether two things are equal and returns `TRUE` or `FALSE`.

```
x == 1.2     # is x equal to 1.2?
```

```
[1] TRUE
```

```
x == pi    # is x equal to pi (~3.14)?
```

```
[1] FALSE
```

install.packages() vs library()

	`install.packages()`	`library()`
What it does	Downloads the package to your computer	Loads it into the current session
When to run	**Once per computer**	**Every session**
In a script?	Never; do it in the Console	Yes: at the top

> **Warning**
>
> Installed once ≠ available always. You must call `library()` at the start of every new R session.

Data frames

A **data frame** is a two-dimensional table:

- Each **row** is one case/observation
- Each **column** is one variable
- Columns can each hold a **different type** (numeric, character, logical, factor, etc.)
- Within a column, every value is the same type

This is what distinguishes a data frame from a **matrix** (all values must be the same type) and from a **vector** (one-dimensional).

```
glimpse(penguins)  # see the structure: rows, columns, types
```

```
Rows: 344
Columns: 8
$ species           <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
$ island            <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
$ bill_length_mm    <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, ~
$ bill_depth_mm     <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, ~
$ flipper_length_mm <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186~
$ body_mass_g       <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, ~
$ sex               <fct> male, female, female, NA, female, male, female, male~
$ year              <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

Named vs positional arguments

Functions take **arguments**. You can pass them **positionally** (by order) or **by name**.

```
round(3.14159, 2)              # positional: x first, digits second
```

```
[1] 3.14
```

```
round(x = 3.14159, digits = 2) # named
```

```
[1] 3.14
```

```
round(digits = 2, x = 3.14159) # named; shows that order doesn't matter
```

```
[1] 3.14
```

Named arguments are safer and more readable, but both styles are correct R.

Arithmetic and vectors

R operates **element-wise** on vectors of the same length:

```
c(1, 2, 3) + c(10, 20, 30)
```

```
[1] 11 22 33
```

```r
c(2, 4, 6) * 3
```

```
[1]  6 12 18
```

Key operators to know:

Expression	What it does
`%%`	Modulo (remainder after division)
`%/%`	Integer division
`^`	Exponent

```r
17 %% 5  # remainder when 17 is divided by 5
```

```
[1] 2
```

```r
17 %/% 5 # how many times does 5 go into 17?
```

```
[1] 3
```

Practice: predict the output

Write your answer before clicking.

(a) `sqrt(81)`

9

(b) `c(3, 6, 9) - c(1, 2, 3)`

2 4 6: element-wise subtraction

(c) `10 %% 3`

1: $10 = 3 \times 3 + 1$

(d) `seq(0, 12, by = 3)`

0 3 6 9 12

(e) `length(c("alpha", "beta", "gamma", "delta"))`

4: four elements in the vector

(f) `mean(c(5, 10, 15, 20), na.rm = TRUE)`

12.5: (5+10+15+20)/4

NA and na.rm

`NA` in R means "not available", i.e., a missing value. Almost any calculation with `NA` returns `NA`:

```
mean(c(2, 4, NA, 8))                # NA
```

```
[1] NA
```

```
mean(c(2, 4, NA, 8), na.rm = TRUE) # drop NAs first
```

```
[1] 4.666667
```

Any function that summarizes values across a vector, such as `mean()`, `sum()`, `min()`, `max()`, `sd()`, accepts `na.rm = TRUE`.

Grammar of graphics

Mapping vs setting aesthetics

	Inside `aes()`	Outside `aes()`
What it does	**Maps** a variable to a visual property	**Sets** a fixed visual property
Changes by row?	Yes; different rows get different values	No; every point gets the same value
Example	`aes(color = species)`	`color = "red"`

```
# color varies by species (mapping):
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(aes(color = species))

# every point is red (setting):
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(color = "red")
```

> ⚠ Warning
>
> `geom_point(aes(color = "red"))` maps the *string* `"red"` as if it were a variable. You get one default color and a legend that says "red". **Set constants outside `aes()`.**

Glyphs

In the **grammar of graphics**, a **glyph** is the basic mark that represents one case.

- In a scatterplot: one dot per row
- In a bar chart: one bar segment per group
- In a line plot: one point connected by a line segment

The glyph is what `geom_*` functions draw. Each row of data produces one glyph (or contributes to one group glyph).

Choosing the right geom

Goal	Appropriate Geom
Distribution of one quantitative variable	`geom_histogram()` or `geom_density()`
Distribution of one categorical variable	`geom_bar()`
Relationship between two quantitative variables	`geom_point()`
Trend over time or ordered x	`geom_line()`
Relationship between quant and categorical	`geom_boxplot()`
Counts of two categoricals	`geom_bar()` + fill + position

Zooming: coord_cartesian() vs xlim()

Both zoom in on a plot, but the difference matters when you have fitted curves:

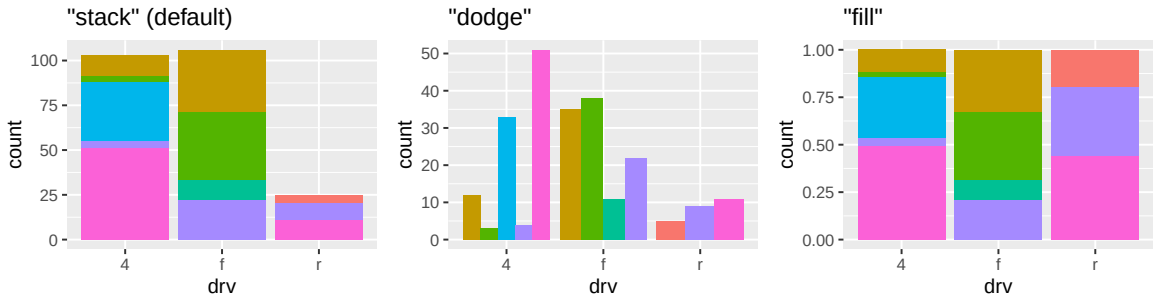
- `xlim(0, 50)` **drops** data outside the range before any computation, which changes what a smoother sees
- `coord_cartesian(xlim = c(0, 50))` only adjusts the **viewport**. Here, all data is still used (you're just zooming in).

```r
# Safer zoom since it doesn't affect statistics:
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth() +
  coord_cartesian(xlim = c(1.5, 4))
```

> **Tip**
>
> Default: use `coord_cartesian()` when you want to zoom without affecting calculations.

Position adjustments

With `geom_bar(aes(x = drv, fill = class))`:



Position	What it does
`"stack"`	Bars stacked; total height = total count
`"dodge"`	Bars side by side within each group
`"fill"`	Bars all same height; shows **proportions**

`"fill"` is best when you want to **compare proportions** across groups.

Facets

Faceting splits the plot into panels, one per level of a variable.

```r
facet_wrap(~ class)    # one variable; auto-wraps into a grid
facet_grid(drv ~ year) # drv in ROWS, year in COLUMNS
```

Key rules for `facet_grid(row_var ~ col_var)`:

- Left of `~` → **rows**
- Right of `~` → **columns**
- Number of panels = levels(row_var) × levels(col_var)
- By default all panels **share the same axis scales**

```
# 3 drv values × 2 year values = 6 panels
ggplot(mpg, aes(x = hwy)) +
  geom_histogram(bins = 15) +
  facet_grid(drv ~ year)
```

Practice: ggplot debugging

What is wrong with this code? There are two separate problems.

```
ggplot(diamonds, aes(x = carat, y = price))
  geom_point()
```

Problem 1: The `ggplot()` line is missing a trailing `+`. Without it, R treats the two lines as separate, unrelated expressions.

Problem 2: If `tidyverse` (or `ggplot2`) was never loaded with `library()` in this session, `geom_point` won't be found, even if the package is installed.

Fixed:

```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_point()
```

Practice: skewed data

A scatterplot of `msleep` with `bodywt` on the x-axis and `sleep_total` on the y-axis shows almost all points crammed into the bottom-left corner, with one or two animals far to the right.

(a) Why does this happen?

`bodywt` is extremely right-skewed, i.e., a few very large animals (elephants, giraffes) pull the x-axis far right, compressing everything else into the left.

(b) Write one line of code to fix it:

```
+ scale_x_log10()
```

A log scale spaces the values multiplicatively so the small animals spread out instead of piling up.

dplyr verbs

The six core verbs

Verb	What it does	Rows changed?	Columns changed?
`filter()`	Keep rows matching a condition	Yes; fewer rows	No
`select()`	Keep (or drop) columns	No	Yes; fewer columns
`mutate()`	Add or modify a column	No	Yes; more/changed columns
`arrange()`	Sort rows	Yes; reordered	No
`group_by()`	Tag groups for later	No visible change	No
`summarise()`	Collapse to one row per group	Yes; fewer rows	Yes; new summary columns

`group_by()` + `summarise()` always go together.

Reduction functions

A **reduction** (or summary) function takes a whole column (vector) and returns **one value**:

```
mean(c(10, 20, 30))
sum(c(10, 20, 30))
n() # only works inside summarise(): counts rows in the group
```

Compare to a **transformation** function — it returns one value *per row*:

```
c(10, 20, 30) * 2  # three inputs, three outputs
```

```
[1] 20 40 60
```

```
log(c(10, 20, 30)) # still three outputs
```

```
[1] 2.302585 2.995732 3.401197
```

Use reductions in `summarise()`. Use transformations in `mutate()`.

Pipeline order matters

The pipe (`|>`) feeds the output of one step into the next. **Order determines what each step sees.**

> ⚠ Warning
>
> You cannot filter on a column that `summarise()` already collapsed away. Always `filter()` first when the condition is on individual rows.

```
# CORRECT: filter rows first, then summarise
penguins |>
  filter(body_mass_g > 4000) |>
  summarise(avg = mean(body_mass_g))

# WRONG: summarise removes the rows - nothing left to filter
penguins |>
  summarise(avg = mean(body_mass_g)) |>
  filter(body_mass_g > 4000) # body_mass_g no longer exists!
```

Reading code: anatomy of a pipeline

```
mpg |>
  filter(class == "compact") |>
  group_by(manufacturer) |>
  summarise(avg_hwy = mean(hwy))
```

Be able to identify each piece:

- **Data table:** `mpg`
- **Functions:** `filter`, `group_by`, `summarise`, `mean`, `==` (operator)
- **Variables (columns):** `class`, `manufacturer`, `hwy`, and the new `avg_hwy`
- **Constants:** `"compact"` (a character string literal)
- **Verb that keeps only some rows:** `filter()`

Practice: write a pipeline

Write a pipeline on `penguins` that:

1. Keeps only rows where `sex` is not `NA`

2. Groups by `species`
3. Reports the count (`n`) and mean flipper length (`avg_flipper`) per species
4. Sorts from longest to shortest average flipper

Try it before looking.

```
penguins |>
  filter(!is.na(sex)) |>
  group_by(species) |>
  summarise(n = n(), avg_flipper = mean(flipper_length_mm, na.rm = TRUE)) |>
  arrange(desc(avg_flipper))
```

```
# A tibble: 3 x 3
  species       n avg_flipper
  <fct>     <int>       <dbl>
1 Gentoo      119        217.
2 Chinstrap    68        196.
3 Adelie      146        190.
```

Practice: pipes and nested calls

Nested → pipe: Rewrite with `|>`, one step per line.

```
head(arrange(filter(mpg, year == 2008), desc(hwy)), 3)
```

```
mpg |>
  filter(year == 2008) |>
  arrange(desc(hwy)) |>
  head(3)
```

The innermost call becomes the first step. `head()` is outermost, so it goes last.

Pipe → nested: Rewrite without the pipe.

```
penguins |>
  filter(!is.na(bill_length_mm)) |>
  summarise(avg_bill = mean(bill_length_mm))
```

```
summarise(filter(penguins, !is.na(bill_length_mm)), avg_bill = mean(bill_length_mm))
```

The over-grouping pitfall

```r
penguins |>
  group_by(species, island, sex) |>
  summarise(avg_mass = mean(body_mass_g, na.rm = TRUE), .groups = "drop") |>
  arrange(desc(avg_mass)) |>
  head(3)
```

```
# A tibble: 3 x 4
  species island sex    avg_mass
  <fct>   <fct>  <fct>     <dbl>
1 Gentoo  Biscoe male      5485.
2 Gentoo  Biscoe female    4680.
3 Gentoo  Biscoe <NA>      4588.
```

The top row has the highest average, but how many penguins is it based on?

Grouping by three variables (species + island + sex) can produce groups with very few penguins. A high average from 2 penguins is not trustworthy.

Fix: always add `n = n()` inside `summarise()` so you can see the group size.

Tidy data

Wide vs narrow (long)

The **same data** can be stored in two shapes:

Wide: one row per student, one column per exam:

```
student   midterm   final
Ava            88      91
Ben            72      85
Cy             95      90
```

Narrow (long): one row per measurement:

```
student   exam       score
Ava       midterm       88
Ava       final         91
Ben       midterm       72
Ben       final         85
Cy        midterm       95
Cy        final         90
```

Wide has **3 rows** and **3 columns**. Narrow has **6 rows** and **3 columns**. They contain the same data, just in a different shape.

pivot_longer() and pivot_wider()

```r
# Wide to narrow (long)
wide_scores |>
  pivot_longer(
    cols = c(midterm, final), # which columns to stack
    names_to  = "exam",       # new column holding the old names
    values_to = "score"       # new column holding the values
  )

# Narrow to wide
narrow_scores |>
  pivot_wider(
    names_from  = exam, # column whose values become new column names
    values_from = score # column whose values fill the new cells
  )
```

Memory trick: `pivot_longer()` makes the table *taller* (more rows). `pivot_wider()` makes it *wider* (more columns). They are exact inverses.

Practice: identify the pivot

(a) You have one row per country and columns `pop_2000`, `pop_2010`, `pop_2020`. You want to plot population over time. Should you `pivot_longer()` or `pivot_wider()`?

`pivot_longer()`: the years are trapped in column names and need to become a `year` variable so you can put it on the x-axis.

(b) You have a long table with one row per (student, subject) pair. You want to compute each student's math score minus their reading score. Should you `pivot_longer()` or `pivot_wider()`?

`pivot_wider()`: put each subject in its own column so that math and reading are side by side in the same row, enabling a `mutate(diff = math - reading)`.

Putting it together

ggplot template

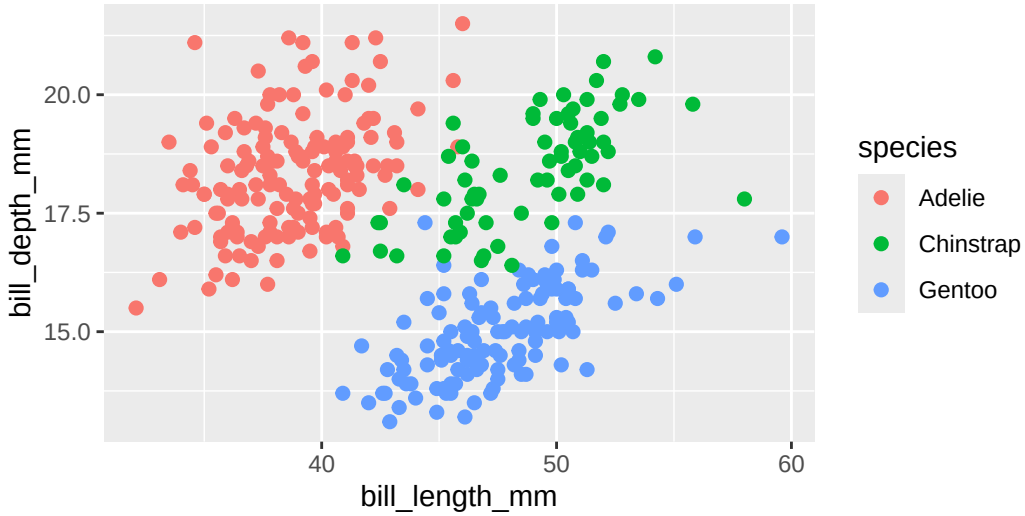
```
ggplot(DATA, aes(x = XVAR, y = YVAR)) +
  geom_SOMETHING(aes(MAPPED_AESTHETICS), FIXED_AESTHETICS)
```

Mapped (inside `aes()`): changes row by row based on a column **Fixed** (outside `aes()`): same for every point/bar/line

Practice: Write a scatterplot of `penguins` with `bill_length_mm` on x, `bill_depth_mm` on y, points **colored by** `species`, and a **fixed** size of 2.

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(aes(color = species), size = 2)
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



`color = species` is a mapping → inside `aes()`. `size = 2` is a constant → outside `aes()`.

Quick-reference: things to know cold

R basics

- `<-` is assignment; `==` is equality test
- `install.packages()` once; `library()` every session
- Data frame: 2D, columns can have different types
- `na.rm = TRUE` to ignore NAs in summary functions

ggplot

- `aes()` maps data; constants go outside `aes()`
- `geom_histogram` for distribution, `geom_point` for scatter, `geom_bar` for counts
- `coord_cartesian()` to zoom without dropping data
- `facet_grid(row ~ col)`: rows left of `~`, columns right
- `position = "fill"` for proportions

dplyr

- `filter` rows, `mutate` columns, `summarise` collapses with `group_by`
- `filter()` must come before `summarise()` if the condition is on individual rows
- Always add `n = n()` to summaries so you know the group size
- `arrange(desc(...))` for descending order

Tidy data

- `pivot_longer()`: wide to narrow (more rows)
- `pivot_wider()`: narrow to wide (more columns)

Good luck tomorrow

Exam

Thursday June 11, 9:35 a.m.

Bring a pen or pencil.

If you finish early, double-check your code writing answers.

If something is unclear during the exam, please raise your hand.